

Lambda KV-Cache Coprocessor

Micro-Architecture & Programming Specification

Longhorn Silicon (Team A71) — SSCS Chipathon 2026, Track D

`lambda_kv_coproc` · GF180MCU core-only macro

Revision A — generated from RTL at signoff

This document is the practical reference for *talking to* the hardened `lambda_kv_coproc` block over its four-wire SPI link: pinout, clocking, the byte-level protocol, the address map and data formats, the status/decision bit-fields, the compress-step state machine, and a complete worked transaction. All values are taken directly from the RTL as hardened (`src/lambda_kv_coproc.sv`, `src/spi_loader.sv`); the hardened tile is $D=2$, $L=2$.

Contents

1	Overview and interfaces	1
1.1	Top-level ports	2
1.2	Internal block map	2
2	Clocking, reset, and timing budget	3
3	SPI protocol	3
4	Address map and data formats	4
4.1	Status / decision / keep bit-fields	4
5	Compress-step state machine	4
6	Address decode	5
7	Programming sequence (worked example)	5
8	Caveats and known issues	6

1 Overview and interfaces

The coprocessor performs one *compress step* per invocation: the host streams L value tokens (D fp16 channels each) and L per-token attention masses in over SPI; three engines then run and the host reads the results back. The engines are:

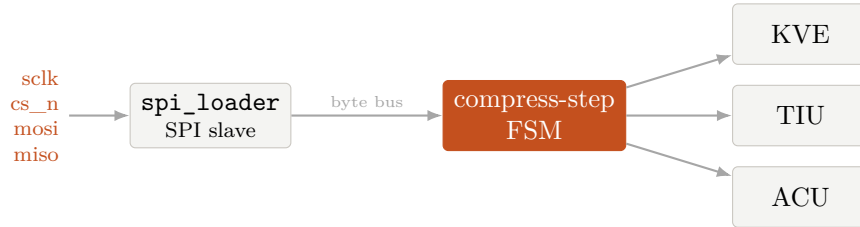
- **KVE** — Walsh–Hadamard rotation + per-token amax + INT3 quantization of each value token (codes, scale, reconstruction).
- **TIU** — H2O importance accumulation, a keep-tier bitmap, and a single eviction victim.
- **ACU** — a divide-free per-tile precision gate.

1.1 Top-level ports

Port	Dir	Function
clk	in	core clock (see §2)
rst_n	in	active-low synchronous-release reset
spi_sclk	in	SPI serial clock (host)
spi_cs_n	in	SPI frame select, active low
spi_mosi	in	host → chip data
spi_miso	out ¹	chip → host data
obs0..obs3	out ¹	status taps: busy, done, err, gate_fp16
VDD/VSS	pwr	5 V core power / common ground

Build parameters (hardened values): $D=2$ (value channels/token, power of two), $L=2$ (cache slots/-tokens per step), $ADDR_WIDTH=16$. Derived constants below assume these values; where a formula helps, L and D are kept symbolic.

1.2 Internal block map



The `spi_loader` decodes SPI frames into a byte-addressed bus (`bus_addr`, `bus_wdata`, `bus_we`, `bus_re`, `bus_rdata`) plus a `start` pulse; it feeds `status_in` back to the host. The compress-step FSM owns the buffers and sequences the engines. Interfaces:

Engine (module)	Inputs from FSM	Outputs to FSM
KVE	<code>in_vec[16D]</code> , <code>dec_idx</code>	<code>out_codes[8D]</code> , <code>out_scale</code> , <code>dec_rot_f16</code>
TIU	<code>acc_*</code> , <code>ld_*</code> , <code>evict_req</code> , <code>token_importance_unit</code> , <code>tier_threshold=48</code>	<code>evict_valid/slot</code> , <code>tier_keep</code> , <code>busy</code>
ACU	<code>s_valid/data/last</code>	<code>d_valid</code> , <code>d_fp16</code>
precision_controller		

The KVE is combinational; TIU and ACU are clocked and handshaked by the FSM.

¹`spi_miso` and `obs0..obs3` are physically bidirectional pads (no output-only digital pad exists in the GF180 IO library) used as outputs.

2 Clocking, reset, and timing budget

Core clock. The block is hardened at a **300 ns period** (≈ 3.33 MHz); this bound is set by the combinational Walsh–Hadamard value path and is the maximum core-clock frequency. Timing closes with zero setup/hold violations at this period.

Reset. `rst_n` is active low with synchronous release. Hold it low for several core clocks after power-good; keep `spi_cs_n` high (idle) during reset. On release the FSM is in `S_IDLE` with all status bits clear.

SPI vs. core clock (important). The SPI inputs are *oversampled in the core clock domain* through a 3-flop synchronizer with rising-edge detection; there is no dedicated async SPI front end. Correct operation therefore requires

$$f(\text{spi_sclk}) \ll f(\text{clk}), \quad T_{\text{sclk-phase}} \gtrsim 4T_{\text{clk}}.$$

In practice budget ≥ 8 **core clocks per SPI bit**. With the core at its 3.33 MHz ceiling this caps `spi_sclk` at roughly **400 kHz**; run slower (100–250 kHz) for margin. `spi_cs_n` must stay low for the whole frame and be sampled by at least two core clocks at each edge.

At-a-glance status. Independently of SPI, the observation pins mirror the status flags live: `obs0`=busy, `obs1`=done, `obs2`=err, `obs3`=gate_fp16 — useful on a logic analyzer or LEDs during bring-up.

3 SPI protocol

Mode 0 (CPOL=0, CPHA=0), MSB-first. A frame is delimited by `spi_cs_n` low $\rightarrow$ high. Data changes while `spi_sclk` is low and is sampled by the slave on the *rising* edge; the host samples `spi_miso` in the high phase. MISO is driven combinatorially and is valid for the whole byte, so the host may sample early in the high phase.

Frame layout.

Byte	Field	Notes
0	CMD	command (below)
1	ADDR[15:8]	big-endian address, high byte first
2	ADDR[7:0]	
3..n	DATA	payload; address auto-increments per byte

Command set.

Value	Command	Behaviour
0x01	WRITE	write DATA bytes starting at ADDR (auto-inc)
0x02	READ	return DATA bytes from ADDR on MISO (auto-inc)
0x03	START	pulse start ; run one compress step (no ADDR/DATA)
0x04	STATUS	MISO returns the STATUS register directly

For READ, after CMD+2 address bytes the host clocks n dummy bytes (e.g. 0x00) on MOSI and captures the returned bytes on MISO; the internal address advances at each byte boundary, so a single frame streams a whole region. **START** is a one-byte frame. Reading STATUS is normally done with READ @ 0x0001 (below); CMD_STATUS is an equivalent shortcut.

4 Address map and data formats

16-bit byte address space; multi-byte fields are **little-endian on the wire** (low byte first). Element ordering is token-major: token 0's D channels, then token 1, etc. **fp16** = IEEE half precision.

Address	Region	Size (bytes)	Dir
0x0300	W — per-token attention mass (u8)	L	write
0x0400	V — value tokens (fp16)	$2LD$	write
0x0800	CODES — rotated INT3 codes (int8)	LD	read
0x0A00	SCALE — per-token scale (fp16)	$2L$	read
0x0C00	VHAT — rotated reconstruction (fp16)	$2LD$	read
0x0001	STATUS	1	read
0x0002	DECISION	1	read
0x0003	KEEP — keep-tier bitmap	1	read

For the hardened tile ($D=2, L=2$): W=2 B, V=8 B, CODES=4 B, SCALE=4 B, VHAT=8 B. V byte order at 0x0400: token0-ch0 low, token0-ch0 high, token0-ch1 low, token0-ch1 high, token1-ch0 low, ...

4.1 Status / decision / keep bit-fields

STATUS (0x0001, read-only):

bit 7	6	5	4	3	2	1	0
0	err	gate_fp16	0	0	0	done	busy

DECISION (0x0002, read-only):

bit 7	bits $[\lceil \log_2 L \rceil - 1 : 0]$
gate_fp16	evict_slot

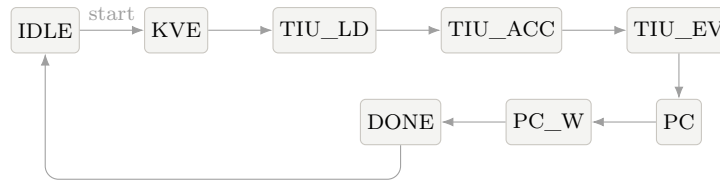
For $L=2$: bit 7 = gate_fp16, bit 0 = evict_slot (bits 6..1 = 0).

KEEP (0x0003, read-only): low L bits are the keep-tier bitmap; bit s is set when slot s 's accumulated mass $\geq$ tier_threshold (=48). Upper bits read 0.

5 Compress-step state machine

On START the FSM leaves S_IDLE (asserting busy) and walks the sequence below, ending in S_DONE (assert done, deassert busy) and returning to idle. Encodings are the RTL's 4-bit state values.

Enc	State	Action
0	S_IDLE	wait for start ; clear done/err .
1	S_KVE	for each token, sweep D channels: latch INT3 code and fp16 reconstruction; capture per-token scale at channel 0.
2	S_TIU_LD	install L slots into the TIU.
3	S_TIU_ACC	accumulate each slot's attention mass.
4	S_TIU_EV	request eviction victim; latch keep-tier bitmap.
5	S_PC	stream the masses into the ACU as the score tile.
6	S_PC_W	wait for the ACU decision; latch gate_fp16 .
7	S_DONE	assert done , deassert busy ; $\rightarrow$ idle.



Error / timeout. S_TIU_EV bounds the eviction handshake and proceeds even if it times out. S_PC_W bounds the ACU handshake; if the decision does not arrive within its window it sets **err** (STATUS bit 6) and goes to S_DONE. A run with **err**=1 means the ACU did not respond; **done** still asserts so the host's poll terminates.

6 Address decode

All decode is combinational in the coprocessor:

- **Write:** **bus_we** writes **W** when **bus_addr** $\in [0x0300, 0x0300+L)$ and **V** when $\in [0x0400, 0x0400+2LD)$; for **V** the low address bit selects the low/high **fp16** byte.
- **Read mux:** 0x0001 $\rightarrow$ STATUS, 0x0002 $\rightarrow$ DECISION, 0x0003 $\rightarrow$ KEEP, CODES/SCALE/VHAT from their windows; any other address reads 0x00.

7 Programming sequence (worked example)

One full compress step for $D=2, L=2$. Each line is one SPI frame (CS low ... high). Bytes are hex; -- marks captured MISO bytes.

```

; 1. write attention masses W (2 bytes) at 0x0300
WRITE 0x0300  m0 m1                      ; CMD=01 ADRH=03 ADRL=00 DATA=m0,m1

; 2. write value tokens V (fp16 LE, 8 bytes) at 0x0400
WRITE 0x0400  t0c0L t0c0H t0c1L t0c1H
              t1c0L t1c0H t1c1L t1c1H

; 3. kick one compress step

```

```

START                                     ; CMD=03 (no address/data)

; 4. poll STATUS until done (bit1), bounded; check err (bit6)==0
loop: READ 0x0001 -> s                   ; CMD=02 ADRH=00 ADRL=01, 1 dummy byte
      until (s & 0x02)                   ; done
      assert (s & 0x40) == 0             ; err clear

; 5. read the decision + keep bitmap
READ 0x0002 -> d                         ; bit7=gate_fp16, bit0=evict_slot (L=2)
READ 0x0003 -> keep                      ; bits[1:0] = keep-tier per slot

; 6. read the compressed records
READ 0x0800 -> c0 c1 c2 c3               ; CODES (L*D = 4 int8, range [-4,3])
READ 0x0A00 -> s0L s0H s1L s1H           ; SCALE (L fp16)
READ 0x0C00 -> h0L h0H ... h3L h3H       ; VHAT (L*D fp16)

```

A READ frame sends CMD=02, the two address bytes, then n dummy MOSI bytes while capturing n MISO bytes; the address auto-increments, so the whole region streams in one frame.

8 Caveats and known issues

- **Reserved low addresses 0x0000–0x0005.** The loader keeps vestigial local CSR shadows here from an earlier datapath. A WRITE to 0x0000 pulses **start** (bit0) and latches **precision_sel** (bits [2:1]); writes to 0x0002–0x0005 land in unused local registers. **Do not write 0x0000–0x0005**; use CMD_START to start. (Reads at 0x0001/0x0002/0x0003 return the coprocessor STATUS/DECISION/ KEEP as documented.)
- **Loader front end.** The SPI slave oversamples in the core domain and is documented in-RTL as a pre-tapeout skeleton (no metastability-hardened async front end). Respect the **spi_sclk** $\ll$ clk budget of §2.
- **Scope.** Verified at the RTL + physical-signoff level (self-checking SPI test + LVS). No gate-level sim; the SPI test is a targeted deterministic check, not exhaustive coverage. See docs/VERIFICATION.md.

Source of truth: `src/lambda_kv_coproc.sv`, `src/spi_loader.sv` on main. Functional model: `src/tb_coproc_spi.sv` (make `sim-coproc`).